

Esercizio 2: Server Linux (Powershell e Processi)

In questo laboratorio, userai la riga di comando Linux per identificare i server in esecuzione su un dato computer.

- Parte 1 - **Server**
- Parte 2 - Usare **Telnet** per Testare i Servizi TCP



Parte 1: Server

I server sono essenzialmente programmi scritti per fornire informazioni specifiche su richiesta. I client, che sono anch'essi programmi, contattano il server, inoltrano la richiesta e attendono la risposta del server. Possono essere utilizzate molte diverse tecnologie di comunicazione **client-server**, la più comune delle quali sono le reti IP. Questo laboratorio si concentra su server e client basati su rete IP.

Accediamo alla VM CyberOps Workstation come **analyst** e per accedere alla riga di comando, fai clic sull'icona del **terminale** situata nel Dock, nella parte inferiore dello schermo della VM. Si apre l'emulatore di terminale.

Molti programmi diversi possono essere in esecuzione su un dato computer, specialmente un computer con sistema operativo Linux. Molti programmi vengono eseguiti in **background**, quindi gli utenti potrebbero non rilevare immediatamente quali programmi sono in esecuzione su un dato computer. In Linux, i programmi in esecuzione sono anche chiamati processi.

Nota: L'output del comando ps differirà perché si baserà sullo stato della tua VM CyberOps Workstation.



Utilizziamo il comando **ps** per visualizzare tutti i programmi in esecuzione in background:

```
[analyst@secOps ~]$ sudo ps -elf
[sudo] password for analyst:
F S UID        PID  PPID  C  PRI  NI ADDR SZ WCHAN  STIME TTY          TIME CMD
4 S root         1     0   0  80   0 -  2250 Sys_ep Feb27 ?      00:00:00 /sbin/init
1 S root         2     0   0  80   0 -    0 kthrea Feb27 ?      00:00:00 [kthreadd]
1 S root         3     2   0  80   0 -    0 smnpboo Feb27 ?      00:00:00 [ksoftirqd/0]
1 S root         5     2   0  60 -20 -    0 worker Feb27 ?      00:00:00 [kworker/0:0H]
1 S root         7     2   0  80   0 -    0 rcu_gp Feb27 ?      00:00:00 [rcu_preempt]
1 S root         8     2   0  80   0 -    0 rcu_gp Feb27 ?      00:00:00 [rcu_sched]
1 S root         9     2   0  80   0 -    0 rcu_gp Feb27 ?      00:00:00 [rcu_bh]
1 S root        10     2   0 -40 -    0 smnpboo Feb27 ?      00:00:00 [migration/0]
1 S root        11     2   0  60 -20 -    0 rescue Feb27 ?      00:00:00 [lru-add-drain]
5 S root        12     2   0 -40 -    0 smnpboo Feb27 ?      00:00:00 [watchdog/0]
1 S root        13     2   0  80   0 -    0 smnpboo Feb27 ?      00:00:00 [cpuhp/0]
5 S root        14     2   0  80   0 -    0 devtmp Feb27 ?      00:00:00 [kdevtmpfs]
1 S root        15     2   0  60 -20 -    0 rescue Feb27 ?      00:00:00 [netns]
1 S root        16     2   0  80   0 -    0 watchd Feb27 ?      00:00:00 [khungtaskd]
1 S root        17     2   0  80   0 -    0 oom_re Feb27 ?      00:00:00 [oom_reaper]
<some output omitted>
```



Perché è stato necessario eseguire ps come root (premettendo il comando con sudo)?

Con **sudo**, abbiamo inclusi alcuni processi del kernel che potrebbero non essere visibili altrimenti. Anche le descrizioni e tutte le informazioni di tutti gli utenti sono più **complete**.

Comando con “sudo”

```
1 I root      882      2   0  80   0 -    0 worker 07:46 ?      00:00:00 [kworker/u17:0]
1 I root      893      2   0  80   0 -    0 worker 07:47 ?      00:00:00 [kworker/u18:1]
4 S root      908     798   0  80   0 -  4958 poll_s 07:48 pts/0    00:00:00 sudo ps -elf
1 S root      910     908   0  80   0 -  4958 poll_s 07:48 pts/1    00:00:00 sudo ps -elf
4 R root      911     910   0  80   0 -  2442 -    07:48 pts/1    00:00:00 ps -elf
[analyst@secOps ~]$ S
```

Comando senza “sudo”

```
1 I root      876      2   0  80   0 -    0 -    07:46 ?      00:00:00 [kworker/u19:1-events_unbound]
1 I root      882      2   0  80   0 -    0 -    07:46 ?      00:00:00 [kworker/u17:0-events_unbound]
1 I root      893      2   0  80   0 -    0 -    07:47 ?      00:00:00 [kworker/u18:1-events_unbound]
1 I root      916      2   0  80   0 -    0 -    07:49 ?      00:00:00 [kworker/u18:2]
0 R analyst   917     798   0  80   0 -  2442 -    07:49 pts/0    00:00:00 ps -elf
[analyst@secOps ~]$
```

In **Linux**, i programmi possono anche chiamare altri programmi. Il comando **ps** può anche essere usato per visualizzare tale gerarchia di processi. Usa le opzioni **-ejH** per visualizzare l'albero dei processi attualmente in esecuzione dopo aver avviato il webserver **nginx** con privilegi elevati.

Nota: Le informazioni sul processo per il servizio **nginx** sono evidenziate. I tuoi valori PID saranno diversi.

```
[analyst@secOps ~]$ sudo /usr/sbin/nginx
[analyst@secOps ~]$ sudo ps -ejH
[sudo] password for analyst:
  PID  PGID  SID TTY      TIME CMD
    1      1    1  ?        00:00:00 systemd
   167   167  167  ?        00:00:01 systemd-journal
   193   193  193  ?        00:00:00 systemd-udev
   209   209  209  ?        00:00:00 rsyslogd
   210   210  210  ?        00:01:41 java
   212   212  212  ?        00:00:01 ovssdb-server
   213   213  213  ?        00:00:00 start_pox.sh
   224   213  213  ?        00:01:18 python2.7
   214   214  214  ?        00:00:00 systemd-logind
   216   216  216  ?        00:00:01 dbus-daemon
   221   221  221  ?        00:00:05 filebeat
   239   239  239  ?        00:00:05 VBoxService
   287   287  287  ?        00:00:00 ovs-vswitchd
   382   382  382  ?        00:00:00 dhcpcd
   387   387  387  ?        00:00:00 lightdm
   410   410  410 tty7      00:00:10 Xorg
   460   387  387  ?        00:00:00 lightdm
   492   492  492  ?        00:00:00 sh
   503   492  492  ?        00:00:00 xfce4-session
   513   492  492  ?        00:00:00 xfwm4
   517   492  492  ?        00:00:00 Thunar
  1592   492  492  ?        00:00:00 thunar-volman
   519   492  492  ?        00:00:00 xfce4-panel
   554   492  492  ?        00:00:00 panel-6-systray
   559   492  492  ?        00:00:00 panel-2-actions
   523   492  492  ?        00:00:01 xfdesktop
   530   492  492  ?        00:00:00 polkit-gnome-au
   395   395  395  ?        00:00:00 nginx
   396   395  395  ?        00:00:00 nginx
   400   384  384  ?        00:01:58 java
   414   414  414  ?        00:00:00 accounts-daemon
   418   418  418  ?        00:00:00 polkitd
<some output omitted>
```



Come viene rappresentata la gerarchia dei processi da ps?

Troviamo diverse colonne in cui sono presenti le seguenti informazioni:

PID = processor ID

PGID = process group ID

SID = session ID

TTY = teletypewriter (terminale di controllo di un processo)

TIME = tempo

CMD = gerarchia dei processi, utilizza indentazione per ordinare le informazioni

I **server** sono programmi che forniscono servizi specifici (ad esempio un web server fornisce servizi web) e sono spesso avviati all'avvio del sistema. Lo strumento da riga di comando **netstat** è essenziale per l'amministrazione di rete, poiché viene utilizzato per visualizzare le connessioni di rete attive sul computer. Sebbene l'esecuzione di netstat senza opzioni restituisca molte informazioni, è possibile filtrarle per renderle più utili; un esempio è l'uso combinato delle opzioni **-tunap** (per visualizzare connessioni TCP/UDP, ascolto, processi e informazioni numeriche) per identificare e monitorare i server in esecuzione, come evidenziato per il server **Nginx** nell'esempio.

```
[analyst@secOps ~]$ sudo netstat -tunap
[sudo] password for analyst:
Active Internet connections (servers and established)
Proto Recv-Q Send-Q Local Address           Foreign Address         State       PID/Program name
tcp        0      0 0.0.0.0:6633           0.0.0.0:*               LISTEN      257/python2.7
tcp        0      0 0.0.0.0:80            0.0.0.0:*               LISTEN      395/nginx: master
tcp        0      0 0.0.0.0:21            0.0.0.0:*               LISTEN      279/vsftpd
tcp        0      0 0.0.0.0:22            0.0.0.0:*               LISTEN      277/sshd: /usr/bin
tcp6       0      0 :::22                 :::*                    LISTEN      277/sshd: /usr/bin
udp        0      0 192.168.1.15:68       0.0.0.0:*               237/systemd-network
```



Qual è il significato delle opzioni **-t**, **-u**, **-n**, **-a** e **-p** in netstat? (usa **man netstat** per rispondere), l'ordine delle opzioni è importante?

```
NETSTAT(8)                                Linux System Administrator's Manual    NETSTAT(8)

NAME
  netstat - Print network connections, routing tables, interface statistics, masquerade connections, and multicast memberships

SYNOPSIS
  netstat [address_family options] [--tcp|-t] [--udp|-u] [--udplite|-U]
  [--sctp|-S] [--raw|-w] [--l2cap|-2] [--rfcomm|-f] [--listening|-l]
  [--all|-a] [--numeric|-n] [--numeric-hosts] [--numeric-ports] [--numeric-users]
  [--symbolic|-N] [--extend|-e[--extend|-e]] [--timers|-o]
  [--program|-p] [--verbose|-v] [--continuous|-c] [--wide|-W]

  netstat [--route|-r] [address_family options] [--extend|-e[--extend|-e]]
  [--verbose|-v] [--numeric|-n] [--numeric-hosts] [--numeric-ports] [--nu-
```

- t = TCP
- u = UDP
- n = mostra indirizzo e porta in forma numerica
- a = mostra tutte le connessioni sia in ascolto che non in ascolto
- p = mostra il pid del processo che esegue il socket

L'ordine delle opzioni in netstat non è importante.



Basandosi sull'output di `netstat -tunap` mostrato al punto (d): Qual è il protocollo di Livello 4, lo stato della connessione e il PID del processo in esecuzione sulla porta 80, e quale servizio di rete (puramente per convenzione) è probabile che stia utilizzando tale porta TCP?

```
[analyst@secOps ~]$ sudo netstat -tunap
[sudo] password for analyst:
Active Internet connections (servers and established)
Proto Recv-Q Send-Q Local Address           Foreign Address         State       PID/Program name
tcp        0      0 0.0.0.0:6633           0.0.0.0:*               LISTEN      257/python2.7
tcp        0      0 0.0.0.0:80            0.0.0.0:*               LISTEN      395/nginx: master
tcp        0      0 0.0.0.0:21            0.0.0.0:*               LISTEN      279/vsftpd
tcp        0      0 0.0.0.0:22            0.0.0.0:*               LISTEN      277/sshd: /usr/bin
tcp6       0      0 :::22                 :::*                    LISTEN      277/sshd: /usr/bin
udp        0      0 192.168.1.15:68       0.0.0.0:*
```

Il servizio in esecuzione sulla porta 80 utilizza il protocollo **TCP** e il PID è **395**, in ascolto, Il servizio in esecuzione sulla porta 80 supponiamo sia un servizio **HTTP** tipicamente utilizzato da un webserver (**nginx** in questo caso)

I client si connetteranno a una porta e, usando il protocollo corretto, richiederanno informazioni a un server. L'output di `netstat` sopra visualizza alcuni servizi che sono attualmente in ascolto su porte specifiche. Colonne interessanti sono:

- La prima colonna mostra il protocollo di Livello 4 in uso **UDP o TCP**, in questo caso).
- La terza colonna usa il formato **INDIRIZZO PORTA** per visualizzare l'indirizzo IP locale e la porta su cui un server specifico è raggiungibile. L'indirizzo IP 0.0.0.0 significa che il server è attualmente **in ascolto** su tutti gli indirizzi IP configurati nel computer.
- La quarta colonna usa lo stesso formato **socket INDIRIZZO PORTA** per visualizzare l'indirizzo e la porta del dispositivo all'estremità remota della connessione. 0.0.0.0 significa che nessun dispositivo remoto sta attualmente utilizzando la connessione.
- La quinta colonna visualizza lo **stato** della connessione.
- La sesta colonna visualizza l'ID del processo (**PID**) responsabile della connessione. Visualizza anche un nome breve associato al processo.

A volte è utile incrociare le informazioni fornite da **netstat con ps**. Basandosi sull'output del punto precedente, si sa che un processo con PID **395** è associato alla porta TCP 80. La porta 395 è usata in questo esempio. Usa ps e grep per elencare tutte le righe dell'output di ps che contengono PID 395.

Sostituisci 395 con il numero PID della tua particolare istanza di nginx in esecuzione:

```
[analyst@secOps ~]$ sudo ps -elf | grep 395
[sudo] password for analyst:
 1 S root      395      1  0  80   0 - 1829   19:33 ?        00:00:00 nginx: master process /usr/bin/nginx
 5 S http      396     395  0  80   0 - 1866   19:33 ?        00:00:00 nginx: worker process
 0 S analyst   3789    1872  0  80   0 - 1190   19:53 pts/0    00:00:00 grep 395
```

Nell'output sopra, il comando `ps` viene inviato tramite **pipe (|)** al comando grep per filtrare solo le righe contenenti il numero 395. Il risultato sono **tre righe** con ritorno a capo del testo.

La **prima** riga mostra un processo di proprietà dell'utente root (terza colonna), avviato da un altro processo con PID 1 (quinta colonna), alle 19:33 (dodicesima colonna).

La **seconda** riga mostra un processo con PID 396, di proprietà dell'utente http, avviato dal processo 395, alle 19:33.

La **terza** riga mostra un processo di proprietà dell'utente analyst, con PID 3789, avviato da un processo con PID 1872, come comando grep 395.



Il processo PID 395 è nginx. Come si potrebbe concludere questo dall'output sopra?

Basandomi sul output precedente si vede: Il processo **396** (master process), figlio di **395** (worker process) gestisce una comunicazione HTTP ed è avviato dal processo padre.



Cos'è nginx? Qual è la sua funzione?

Nginx è un **web server** ad alte prestazioni, può utilizzare tecnologie come reverse proxy, load balancer e può avere una cache



La seconda riga mostra che il processo 396 è di proprietà di un utente chiamato http e ha il processo numero 395 come processo genitore. Cosa significa? È un comportamento comune?

Il processo master di **Nginx** (pid 395) viene avviato come **ROOT**, che avvia un worker process con privilegi inferiori. Questo aiuta a **rispettare** la regola del minimo privilegio necessario ed è un'operazione comune.

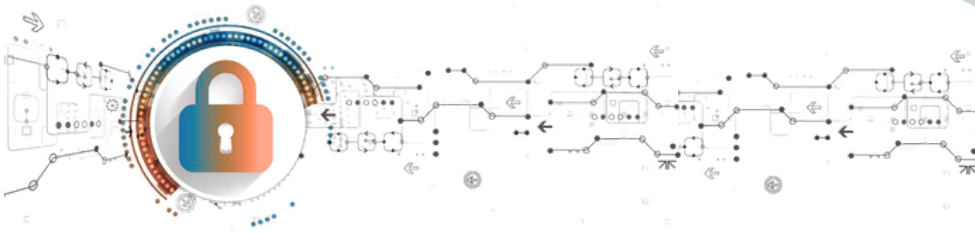


Perché l'ultima riga mostra `grep 395`?

Con **grep** eseguiamo un processo di **filtraggio** per trovare informazioni sul PID **395**. PS mostra tutti i processi mentre "grep" filtra tutti i processi che utilizzano 395 compreso se stesso



FINE Parte 1



Server Linux

In questo laboratorio, userai la riga di comando Linux per identificare i server in esecuzione su un dato computer.

- Parte 1 - **Server**
- Parte 2 - Usare **Telnet** per Testare i Servizi TCP



Parte 2: Usare Telnet per Testare i Servizi TCP

Telnet è una semplice applicazione di **shell remota**. Telnet è considerato insicuro perché non fornisce crittografia. Gli amministratori che scelgono di usare Telnet per gestire remotamente dispositivi di rete e server esporranno le credenziali di accesso a quel server, poiché Telnet trasmetterà i dati della sessione in chiaro. Sebbene Telnet non sia raccomandato come applicazione di shell remota, può essere molto utile per **testare** rapidamente o raccogliere informazioni sui servizi TCP.

Il protocollo Telnet opera sulla porta **23** usando **TCP** per impostazione predefinita. Il client telnet, tuttavia, permette di specificare una porta diversa. Cambiando la porta e connettendosi a un server, il client telnet permette a un analista di rete di valutare rapidamente la natura di un **server** specifico comunicando direttamente con esso.

Nota: Si raccomanda vivamente di usare ssh come applicazione di shell remota invece di telnet.

telnet>_

Nella Parte 1, si è scoperto che **nginx** era in esecuzione e assegnato alla porta 80 TCP. Sebbene una rapida ricerca su internet abbia rivelato che nginx è un server web **leggero**, come potrebbe un analista esserne sicuro? E se un attaccante avesse cambiato il nome di un programma malware in nginx, solo per farlo sembrare il popolare webserver? Usa **telnet** per connetterti all'host locale sulla porta 80 TCP

```
[analyst@secOps ~]$ telnet 127.0.0.1 80
Trying 127.0.0.1...
Connected to 127.0.0.1.
Escape character is '^]'.
```

Premi alcune lettere sulla tastiera. Qualsiasi tasto funzionerà. Dopo aver premuto alcuni tasti, premi **INVIO**. Sotto c'è l'output completo, inclusa l'instaurazione della connessione Telnet e i tasti **casuali** premuti (fdsafsdaf, in questo caso):

```
fdsafsdaf
HTTP/1.1 400 Bad Request
Server: nginx/1.16.1
Date: Tue, 28 Apr 2020 20:09:37 GMT
Content-Type: text/html
Content-Length: 173
Connection: close

<html>
<head><title>400 Bad Request</title></head>
<body bgcolor="white">
<center><h1>400 Bad Request</h1></center>
<hr><center>nginx/1.16.1</center>
</body>
</html>
Connection closed by foreign host.
```

Grazie al protocollo Telnet, è stata stabilita una connessione **TCP** in chiaro, dal client Telnet, direttamente al server nginx, in ascolto su 127.0.0.1 porta **80** TCP. Questa connessione ci **permette** di inviare dati direttamente al server. Poiché nginx è un server web, non comprende la sequenza di lettere casuali inviate ad esso e restituisce un **errore** nel formato di una pagina web.



Perché l'errore è stato inviato come pagina web?

Perché i server da protocollo comunicano tramite risposte **HTTP**

```
[analyst@secOps ~]$ telnet 127.0.0.1 80
Trying 127.0.0.1...
Connected to 127.0.0.1.
Escape character is '^]'.
fskad;fj;osdjf
HTTP/1.1 400 Bad Request
Server: nginx/1.28.0
Date: Mon, 29 Sep 2025 12:54:23 GMT
Content-Type: text/html
Content-Length: 157
Connection: close

<html>
<head><title>400 Bad Request</title></head>
<body>
<center><h1>400 Bad Request</h1></center>
<hr><center>nginx/1.28.0</center>
</body>
</html>
Connection closed by foreign host.
[analyst@secOps ~]$
```



Usa Telnet per connetterti alla porta 68. Cosa succede?

Succede che non ci sono servizi attivi in ascolto sulla porta 68

```
[analyst@secOps ~]$ telnet 127.0.0.1 68
Trying 127.0.0.1...
telnet: Unable to connect to remote host: Connection refused
```



Quali sono i vantaggi nell'uso di Netstat?

Netstat è uno strumento diagnostico di rete molto potente. I suoi principali vantaggi sono:

- **Visibilità delle connessioni:** Permette di vedere tutte le connessioni di rete attive (sia in entrata che in uscita) e su quali porte il sistema è in ascolto.
- **Associazione Processo-Porta:** Con l'opzione -p, consente di identificare esattamente quale programma o servizio sta utilizzando una specifica porta di rete, il che è fondamentale per la risoluzione dei problemi e la sicurezza.
- **Supporto per più protocolli:** Può mostrare informazioni per diversi protocolli, tra cui TCP, UDP e socket UNIX.
- **Flessibilità:** Offre numerose opzioni per filtrare e formattare l'output in base alle necessità dell'amministratore.



Quali sono i vantaggi nell'uso di telnet? è sicuro?

I vantaggi di Telnet come strumento di diagnostica sono:

- Consente di testare rapidamente se un servizio TCP su una porta specifica è in **ascolto e raggiungibile**.
- Ottimo per il **debug** di protocolli basati su testo come **HTTP, SMTP o POP3**.
- Permette di inviare dati **grezzi** a un servizio e osservarne la risposta non filtrata.

No, **non** è considerato un protocollo sicuro perchè tutti i dati, incluse le credenziali (username e password), vengono trasmessi in **chiaro** (senza crittografia). Chiunque sia in grado di intercettare il traffico di rete può leggere facilmente queste informazioni.

Per l'amministrazione remota sicura, si dovrebbe sempre usare **SSH** (Secure Shell), che cripta l'intera sessione di comunicazione